

ConsensusScope: A Teacher-Controlled Safety Routing System for AI-Generated ESL Writing Feedback

You Tingrui¹, Li Jimei^{2*}, Zhang Na², Xiong Ling², Zhang Xiaodan¹, Xiao Shurou¹, and Yang Yue¹

¹Hainan International College, Beijing Language and Culture University

²School of Information Science, Beijing Language and Culture University

yyting2006@163.com, Ljm@blcu.edu.cn, zn17333099816@163.com, lingdonghua99@163.com
zhang_071019@qq.com, 2892676221@qq.com, yangyueblcu@163.com

Abstract

Generative AI can reduce the workload of English essay feedback, but fluent suggestions may still change student meaning, add unsupported content, or over-correct a draft. We present ConsensusScope, an interactive system for safety routing of AI-generated English essay feedback. The system uses each feedback item as the unit of control: it normalizes candidates, extracts deploy-time safety signals, constructs an auditable Feedback Safety Graph, and applies a pilot-informed release policy to route each item to `auto_accept`, `teacher_review`, or `needs_more_evidence`. To make this routing layer directly usable, we instantiate it as a Streamlit/FastAPI demo for single-essay review, batch review, teacher queueing, graph inspection, and report export. We evaluate routing with a stress suite, a controlled perturbation suite derived from public learner corpora, and a 30-item two-teacher pilot. The perturbation suite checks whether known high-risk feedback patterns are consistently held for review. In the diagnostic teacher pilot, FSGR auto-routes 7/30 items, of which 6/7 are teacher-safe; an LLM-as-judge baseline auto-routes 9/30 items, of which 7/9 are teacher-safe. The pilot also obtains 0.694 within-one-point teacher agreement. These results support graph-backed safety routing as a teacher-control layer for AI feedback release.

1 Introduction

Generative AI is becoming a plausible assistant for English essay feedback, a task with a long history in grammatical error correction (GEC) benchmarks (Ng et al., 2014; Bryant et al., 2019), learner-writing corpora (Yannakoudakis et al., 2011; Napoles et al., 2017), and automated writing support (Madnani et al., 2018). The practical appeal is clear:

teachers can obtain grammar, vocabulary, organization, and argument comments at essay scale. The unresolved bottleneck is safe release. A fluent AI comment can be wrong after a modal verb, shift the student’s claim, add unsupported evidence, or give advice that is too broad to show directly to learners. Reviewing every suggestion removes the promised workload reduction; releasing every suggestion risks misleading students.

ConsensusScope formulates this bottleneck as item-level safety routing. Given an anonymized ESL draft and AI feedback candidates, the system routes each item to `auto_accept`, `teacher_review`, or `needs_more_evidence`. The decision unit is a concrete feedback item tied to a target span, local context, suggestion, rationale, and issue type, which lets the system make release decisions before feedback becomes student-facing.¹

The core mechanism is Feedback Safety Graph Routing (Figure 1). ConsensusScope links the target span, AI suggestion, evidence status, active safety dimensions, and final route in an auditable graph. Deploy-time signals include meaning-preservation warnings, evidence-grounding risks, local-edit scope, pedagogical tone and specificity, and model/system reliability signals such as low agreement and parse failures. The graph preserves the evidence path used to release, review, or hold back each item, making the route inspectable by teachers and reusable for later audit.

We evaluate this routing layer with stress checks, a controlled public-corpus perturbation suite, an LLM-as-judge baseline, and a 30-item two-teacher diagnostic pilot. The ev-

¹Code and hosted demo: <https://github.com/yyting2006-ai/ConsensusScope> and <https://demo.consensuscope.cn/>.

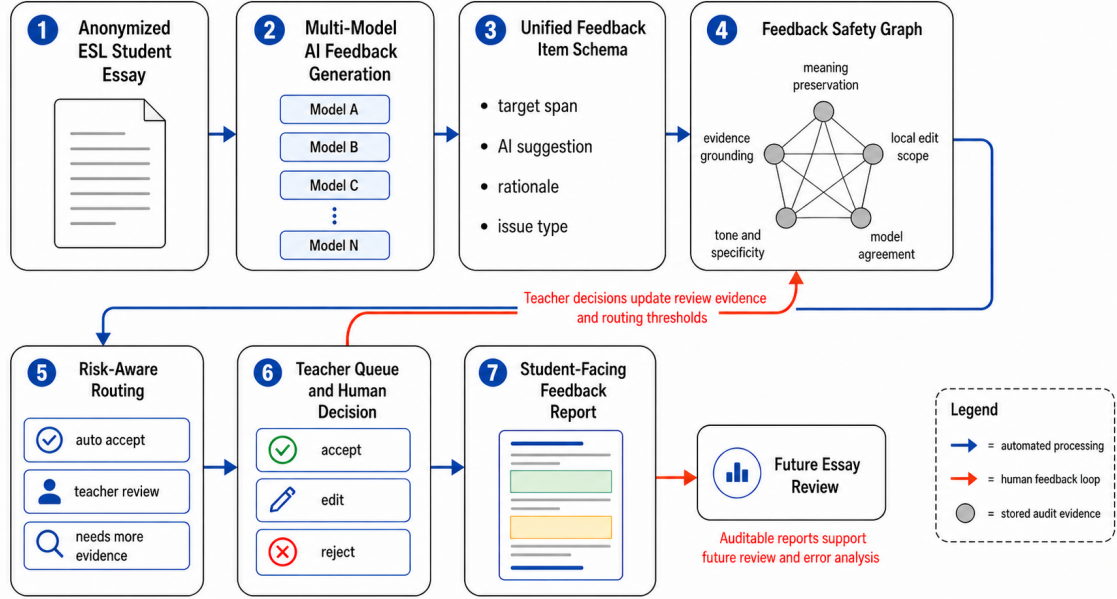


Figure 1: ConsensusScope workflow for safety routing of AI-generated ESL writing feedback. Blue arrows denote automated processing from anonymized essays and multi-model feedback to item-level routing decisions; red arrows denote teacher feedback loops that update review evidence and future audit analysis.

idence focuses on route quality: ConsensusScope can auto-route some low-risk feedback while keeping constructed unsafe items and teacher-marked unsafe items in the review queue. The demo contributes a safety-routing formulation, a Feedback Safety Graph Routing algorithm with a pilot-informed release policy, a usable Streamlit/FastAPI review system, and a reproducible evaluation package for teacher-supervised AI feedback.

2 Related Work

Learner writing and GEC. GEC benchmarks have provided standardized correction tasks and learner-writing data, including CoNLL-2014 (Ng et al., 2014), BEA-2019 W&I+LOCNESS (Bryant et al., 2019), FCE (Yannakoudakis et al., 2011), and JFLEG (Napoles et al., 2017). Earlier shared tasks and corpora such as HOO and NUCLE helped establish learner-text correction as a benchmarked NLP problem (Dale and Kilgarriff, 2011; Dahlmeier et al., 2013). Phrase-level edit extraction and scorer conventions such as M2 and ERRANT further standardized how original/corrected pairs are converted into edit records (Dahlmeier and Ng, 2012; Bryant et al., 2017). These datasets primarily

evaluate correction quality. ConsensusScope uses their correction annotations only as offline evidence for a downstream routing question: whether risky or incorrect feedback candidates remain in a review queue.

Automated writing feedback. Automated writing systems can provide revision support, but classroom use depends on how learners and teachers interpret the feedback (Madnani et al., 2018; Ranalli, 2018). ConsensusScope follows this line of work by keeping teachers in the release loop and making AI feedback inspectable before it becomes student-facing.

LLM reliability and adjudication. Multi-output and judge-based methods can help select answers or evaluate generations (Wang et al., 2023; Zheng et al., 2023; Liu et al., 2023). For educational feedback, release decisions also need item-level safety evidence. Our demo treats model agreement as one observable signal alongside issue type, evidence status, meaning-change warnings, unsupported-claim cues, tone cues, and parse errors.

System demonstrations. Recent NLP demo papers emphasize usable interfaces, public APIs, and diagnostic views, including educational assessment systems (Li et al., 2025),

deployment-oriented assistants (Trienes et al., 2025; Jang et al., 2025), LLM-as-judge tooling (Santillán Cooper et al., 2025), prompt evaluation interfaces (Habba et al., 2025), and trajectory diagnosis systems (Ou et al., 2025). ConsensusScope follows this demo-track pattern but focuses on the narrower teacher-facing question of whether generated writing feedback should be released, reviewed, or held for more evidence.

Table 1 summarizes the task focus of ConsensusScope relative to neighboring system types: it routes each feedback item before it reaches a student and exposes graph evidence behind release or review decisions.

3 Method

ConsensusScope implements Feedback Safety Graph Routing (FSGR), an item-level routing method for deciding whether an AI-generated writing-feedback suggestion can be shown to a student. The workflow has three input-output modules: given an anonymized essay E and feedback candidates $\mathcal{C} = \{c_i\}_{i=1}^n$, Module 1 normalizes each candidate, Module 2 builds graph-backed safety evidence, and Module 3 maps that evidence to a route $r_i \in \{A, V, E\}$, where A denotes automatic release, V denotes teacher review, and E denotes a request for more evidence.

Module 1: unified feedback item. FSGR first normalizes each raw candidate into a stable schema $f_i = (d_i, x_i, z_i, u_i, h_i, t_i, a_i)$: essay id, target span, surrounding context, AI suggestion, rationale, predicted issue type, and optional agreement signal. This makes live OpenAI-compatible feedback, deterministic benchmark candidates, and stored audit records enter the same downstream router. The module output is the comparable feedback item f_i .

Module 2: Feedback Safety Graph. The router extracts deploy-time signals for parse failure, evidence gaps, meaning-change cues, unsupported claims, whole-draft rewriting, tone warnings, teacher-dependent wording, broad target spans, and agreement risk. These signals activate five graph dimensions: meaning preservation, evidence grounding, local edit scope, pedagogical tone and specificity,

and model/system reliability. The graph schema contains nodes for student text, AI suggestion, issue type, evidence status, active safety dimensions, and routing decision; edges encode contains, is_revised_by, activates, and justifies_route. The stored path has the form target span $\rightarrow$ AI suggestion $\rightarrow$ active safety dimension $\rightarrow$ route, so teachers can inspect why an item is released or queued. The same active dimensions are passed to the release policy, linking the visual audit record with the routing decision. The module output is the graph record G_i and its active safety signals.

Module 3: release policy. FSGR computes a bounded risk score and a calibrated review probability,

$$s_i = \min(1, \sum_k \alpha_k b_{ik}), \quad p_i = \sigma(\beta_0 + \beta^\top x_i),$$

where b_i is the deploy-time signal vector and x_i combines risk score and active graph dimensions, following the logistic regression formulation (Cox, 1958). The logistic layer provides transparent score aggregation with pilot-informed, auditable thresholds. High-severity signals and parse/evidence failures can override the probability. The module output is the final route:

$$r_i = \begin{cases} E, & \phi_i = 1, \\ V, & h(b_i) = 1 \vee p_i \geq \tau, \\ A, & \text{otherwise.} \end{cases}$$

Teacher ratings and public-corpus gold corrections are used only for offline calibration and evaluation; they are not available when routing a new item.

4 System Interface

The demo turns FSGR into a teacher-facing review workflow. In single-essay mode, a teacher reviews an anonymized ESL draft and inspects item-level routes. In batch mode, the system prioritizes medium- and high-risk feedback in a teacher queue before exporting a student-facing report. The FastAPI backend exposes the same router for programmatic use, returning the route, risk reasons, calibrated review probability, and graph record.

A typical demonstration starts when a teacher opens an anonymized essay. The

System type	Main object	Typical output	ConsensusScope distinction
GEC / revision tools	Sentence edit	Corrected text	Audits whether feedback should be student-facing
Essay scoring systems	Whole essay	Score or rubric label	Avoids grading and routes item-level AI feedback
LLM-as-judge workflows	Model output	Preference or quality score	Uses graph-backed deploy-time safety signals
Feedback dashboards	Review records	Tables or filters	Connects target span, suggestion, risk, route, and teacher action

Table 1: Comparison of task focus: ConsensusScope targets feedback safety routing with item-level graph evidence.

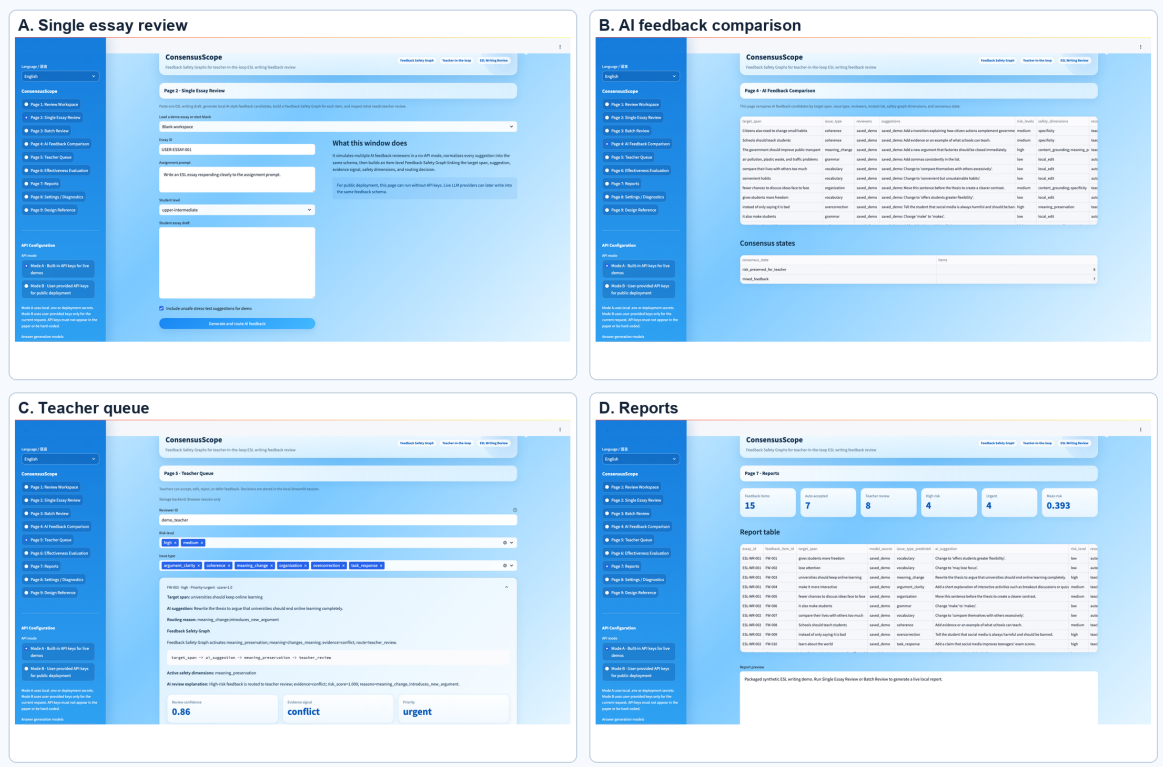


Figure 2: ConsensusScope online demo interface. The current deployed system supports single-essay review, AI-feedback comparison, teacher queueing with graph evidence, and report export.

system displays generated feedback candidates, auto-releases narrow local edits, queues meaning-changing or unsupported suggestions, lets the teacher accept/edit/reject queued items, and exports only approved feedback into a student-facing report. The graph trace remains available for later audit.

The main users are English writing teachers, educational NLP developers, and researchers studying human-in-the-loop feedback. Teachers use the queue to separate low-risk local edits from suggestions requiring professional judgment. Developers use the unified records to debug generators, and researchers can compare routing policies and teacher-aligned diag-

nostics. The system is useful even when the underlying feedback generator changes, because the routing layer only assumes the unified feedback schema.

5 Evaluation

We evaluate ConsensusScope as a review-routing layer. The comparison question is whether FSGR gives a useful safety-efficiency trade-off relative to simpler release policies and an LLM-as-judge router over the same feedback items. We measure unsafe-feedback recall in the review queue, auto-release share, and alignment with teacher-facing diagnostics.

Source	Records	Gold edits	Correct cand.	Risk dist.	Candidates	Auto	Review
JFLEG	1,501	3,938	3,938	7,649	11,587	0.320	0.680
CoNLL-2014	2,624	5,110	5,110	10,016	15,126	0.324	0.676
FCE	33,236	45,969	45,969	90,965	136,934	0.329	0.671
W&I+LOCNESS	38,692	62,384	62,384	123,751	186,135	0.329	0.671
Total	76,053	117,401	117,401	232,381	349,782	0.328	0.672

Table 2: Public learner-corpus routing benchmark. Candidate counts are derived from public gold edits: each edit contributes one correct candidate and one or two constructed risk distractors.

Stress checks and public benchmark. The packaged demo contains 31 stress and expected-route cases covering thesis reversal, unsupported factual claims, whole-essay rewriting, harsh tone, low agreement, and parse failures; the router matches the expected route labels. For a controlled public-corpus perturbation suite, we use learner-correction corpora: JFLEG (Napoles et al., 2017), CoNLL-2014 (Ng et al., 2014), FCE (Yannakoudakis et al., 2011), and W&I+LOCNESS (Bryant et al., 2019). We use edit-distance alignment (Levenshtein, 1966), following edit-centric GEC evaluation (Dahlmeier and Ng, 2012; Bryant et al., 2017). Each public gold edit yields one correct feedback candidate and one or two constructed risk distractors. Thus the 349,782 candidates are derived from 117,401 public gold edits plus 232,381 constructed risk distractors, not newly collected student records or uncontrolled LLM generations.

Teacher diagnostics and routing utility. A blind two-teacher pilot over 30 anonymized AI feedback items checks whether routes align with human judgments on correctness, meaning preservation, student readiness, usefulness, clarity, and direct-release suitability. The router obtains 1.000 recall for teacher-marked review-needed and unsafe items, 0.857 auto-accept precision against teacher-safe items, and 0.694 mean within-one-point teacher agreement across six Likert dimensions. ICC(2,1) is moderate for correctness (0.655) and direct-release suitability (0.616), but lower for clarity (0.154), reflecting teacher disagreement about wording quality.

Table 3 compares routing utility with an LLM-as-judge baseline, an issue-type rule baseline, a score-only cutoff, review-all, and auto-release-all. We do not use this small pilot table to claim that the graph alone improves aggregate accuracy: the score-only cut-

Policy	Auto n	Safe n	Auto safe	Review rec.	Unsafe auto	Trace
FSGR default	7	6	0.857	1.000	0	yes
LLM-as-judge	9	7	0.778	0.947	0	no
Issue-type rule	11	6	0.546	0.789	2	no
Score-only cutoff	7	6	0.857	1.000	0	partial
Review all	0	0	—	1.000	0	no
Auto-release all	30	10	0.333	0.000	15	no

Table 3: Teacher-pilot routing utility over 30 blind-rated feedback items. Auto n is the number automatically released; Safe n is the number judged teacher-safe; Review rec. is recall for teacher-marked review-needed items. Trace indicates whether an exported span-suggestion-dimension-route audit path is available.

Graph signal group	Cases	Match	Routed failure mode
Local edit scope	5	5/5	low-risk release or local override
Meaning preservation	6	6/6	thesis reversal, rewrite, wrong correction
Content grounding	4	4/4	unsupported claim or source injection
Specificity	8	8/8	vague or teacher-dependent advice
Pedagogical tone	2	2/2	harsh student-facing wording
Model agreement	5	5/5	low-agreement release prevention

Table 4: Graph stress-check coverage on 16 packaged cases. Match counts require both expected route and expected risk level to match. Cases may activate multiple graph signals; this table evaluates functional coverage, not statistical significance.

off matches FSGR on the 30-item pilot. The graph contribution is instead traceable routing evidence and coverage of safety failure modes, summarized in Table 4. The LLM-as-judge baseline was run on July 4, 2026 (CST) with deepseek-chat through the configured judge provider and an OpenAI-compatible DeepSeek chat-completions endpoint. It uses temperature 0, max_tokens=240, prompt-enforced JSON output rather than a provider JSON-mode flag, and the same target span, context, suggestion, rationale, and issue type seen by the router, excluding teacher labels and system routes. The client retries failed requests twice; any request or parse failure is mapped to review. No request or parse failures occurred in the reported run. Its JSON output is mapped to auto-release only when safe_to_release=true and needs_teacher_review=false.

The public benchmark should be interpreted as controlled perturbation testing: it verifies that the router behaves consistently on public-corpus-derived corrections and constructed safety distractors. The teacher pilot provides the direct, but small-scale, evidence for teacher-aligned routing utility. For FSGR, a bootstrap interval over the pilot gives a wide 95% interval for auto-release precision against

teacher-safe items (0.50–1.00), while review-needed and unsafe-item recall remain 1.00 in the resamples.

6 Availability and Reproducibility

The repository, live demo, video, and API documentation are available at <https://github.com/yyting2006-ai/ConsensusScope>, <https://demo.consensusscope.cn/>, <https://youtu.be/vp4Kj48bv8Q>, and <https://api.consensusscope.cn/docs>. The code is MIT-licensed; third-party corpora and APIs keep their original terms. The repository includes startup instructions and `scripts/run_public_gec_benchmark.py`; raw corpora and per-item derived outputs are not committed.

7 Ethics and Limitations

ConsensusScope is a teacher-support tool, not an automatic essay scorer or teacher replacement. Users should upload only anonymized student writing. The benchmark validates graph-backed routing with public correction labels and constructed distractors; it does not measure learning gains, time-on-task, or real LLM feedback quality. The teacher pilot is diagnostic, not a classroom study. The hosted demo is intended for anonymized examples and demonstration use. For real classroom deployment, we recommend local installation or institution-approved data handling. If external LLM providers are used for feedback generation or judging, teachers should ensure that student data are anonymized and processed under appropriate institutional policies.

8 Conclusion

ConsensusScope demonstrates safety routing for generative AI English essay feedback: it converts feedback items into graph-backed release decisions so teachers can reduce review burden without giving up control over what students receive.

References

Christopher Bryant, Mariano Felice, Øistein E. Andersen, and Ted Briscoe. 2019. [The BEA-2019 shared task on grammatical error correction](#). In Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications,

pages 52–75, Florence, Italy. Association for Computational Linguistics.

Christopher Bryant, Mariano Felice, and Ted Briscoe. 2017. [Automatic annotation and evaluation of error types for grammatical error correction](#). In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 793–805, Vancouver, Canada. Association for Computational Linguistics.

David R. Cox. 1958. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2):215–232.

Daniel Dahlmeier and Hwee Tou Ng. 2012. [Better evaluation for grammatical error correction](#). In Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 568–572, Montréal, Canada. Association for Computational Linguistics.

Daniel Dahlmeier, Hwee Tou Ng, and Siew Mei Wu. 2013. [Building a large annotated corpus of learner english: The NUS corpus of learner english](#). In Proceedings of the Eighth Workshop on Innovative Use of NLP for Building Educational Applications, pages 22–31, Atlanta, Georgia. Association for Computational Linguistics.

Robert Dale and Adam Kilgariff. 2011. [Helping our own: The HOO 2011 pilot shared task](#). In Proceedings of the 13th European Workshop on Natural Language Generation, pages 242–249, Nancy, France. Association for Computational Linguistics.

Eliya Habba, Noam Dahan, Gili Lior, and Gabriel Stanovsky. 2025. [PromptSuite: A task-agnostic framework for multi-prompt generation](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 254–263, Suzhou, China. Association for Computational Linguistics.

Dongsuk Jang, Ziyao Shanguan, Kyle Tegtmeier, Anurag Gupta, Jan T. Czerwinski, Sophie Chheang, and Arman Cohan. 2025. [MedTutor: A retrieval-augmented LLM system for case-based medical education](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 319–353, Suzhou, China. Association for Computational Linguistics.

Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.

Jiazheng Li, Artem Bobrov, Runcong Zhao, Cesare Aloisi, and Yulan He. 2025. [AERA chat: An interactive platform for automated explainable student answer assessment](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 536–545, Suzhou, China. Association for Computational Linguistics.

Yang Liu, Dan Iter, Yichong Xu, Shuhang Wang, Ruochen Xu, and Chenguang Zhu. 2023. [G-Eval: NLG evaluation using GPT-4 with better human alignment](#). arXiv preprint arXiv:2303.16634.

Nitin Madnani, Jill Burstein, Norbert Elliot, Beata Beigman Klebanov, Diane Napolitano, Slava Andreyev, and Maxwell Schwartz. 2018. [Writing mentor: Self-regulated writing feedback for struggling writers](#). In Proceedings of the 27th International Conference on Computational Linguistics: System Demonstrations, pages 113–117, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Courtney Napoles, Keisuke Sakaguchi, and Joel Tetreault. 2017. [JFLEG: A fluency corpus and benchmark for grammatical error correction](#). In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers, pages 229–234, Valencia, Spain. Association for Computational Linguistics.

Hwee Tou Ng, Siew Mei Wu, Ted Briscoe, Christian Hadiwinoto, Raymond Hendy Susanto, and Christopher Bryant. 2014. [The CoNLL-2014 shared task on grammatical error correction](#). In Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task, pages 1–14, Baltimore, Maryland. Association for Computational Linguistics.

Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. 2025. [AgentDiagnose: An open toolkit for diagnosing LLM agent trajectories](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 207–215, Suzhou, China. Association for Computational Linguistics.

Jim Ranalli. 2018. [Automated written corrective feedback: how well can students make use of it?](#) Computer Assisted Language Learning, 31(7):653–674.

Martín Santillán Cooper, Zahra Ashktorab, Hyo Jin Do, Erik Miehling, Werner Geyer, Jasmina Gajcin, Elizabeth M. Daly, Qian Pan, and Michael Desmond. 2025. [Synthetic data for evaluation: Supporting LLM-as-a-judge workflows with EvalAssist](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 1–11, Suzhou, China. Association for Computational Linguistics.

Jan Trienes, Anastasiia Derzhanskaia, Roland Schwarzkopf, Markus Mühling, Jörg Schlötterer, and Christin Seifert. 2025. [Marcel: A lightweight and open-source conversational agent for university student support](#). In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 181–195, Suzhou, China. Association for Computational Linguistics.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). In International Conference on Learning Representations.

Helen Yannakoudakis, Ted Briscoe, and Ben Medlock. 2011. [A new dataset and method for automatically grading ESOL texts](#). In Proceedings of the 49th Annual Meeting of the Association for Computational

Linguistics: Human Language Technologies, pages 180–189, Portland, Oregon, USA. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging llm-as-a-judge with mt-bench and chatbot arena](#). In Advances in Neural Information Processing Systems.

A Supplementary Implementation Details

This appendix provides compact implementation artifacts for reproducing the routing behavior described in the main paper. The appendix is not used to add new experimental claims; it documents the router, exported fields, and pilot case patterns.

Router pseudocode.

Algorithm A.1: Feedback Safety Graph Router
 Input: normalized item f_i , local context z_i , optional rationale h_i , agreement signal a_i
 Output: route r_i , review probability p_i , risk score s_i , graph G_i

1. Extract deploy-time signals b_i for parse failure, evidence conflict, meaning-change cues, unsupported claims, tone warnings, broad target spans, and agreement risk.
2. Map b_i to the five graph dimensions: meaning preservation, evidence grounding, local edit scope, pedagogical tone and specificity, and model/system reliability.
3. Construct G_i with target-span, suggestion, evidence, dimension, and route nodes.
4. Compute $s_i = \min(1, \sum_k \alpha_k b_{ik})$.
5. Build $x_i = [s_i; \text{active dimensions}]$ and compute $p_i = \sigma(\beta_0 + \beta^\top x_i)$.
6. If parse or grounding failure is active, set $r_i = E$.
7. Else if a high-severity signal is active or $p_i \geq \tau$, set $r_i = V$.
8. Else set $r_i = A$.
9. Return r_i, p_i, s_i , risk reasons, graph path, and JSON records.

The actions E , V , and A denote evidence request, teacher review, and automatic release. High-severity graph signals can override the review score so that clearly unsafe feedback remains in the teacher queue.

Representative routed cases.

Case	Pattern	Route	Diagnostic use
FB-002	Local phrase edit	Auto	Teachers rate safe
FB-003	Thesis reversal	Review	Blocks meaning change
FB-005	Wrong modal edit	Review	Finds unsafe local edit
FB-008	Unsupported claim	Review	Blocks evidence gap

Table A.1: Representative pilot cases. Teacher ratings are offline diagnostics, not deploy-time labels.